

SFTTrainer Parameters

- **model** (`PreTrainedModel` or `PeftModel`) — The model to train, evaluate or use for predictions.
- **args** (`SFTConfig`, *optional*) — The arguments to tweak for training.
- **data_collator** (`DataCollator`, *optional*) — Function to use to form a batch from a list of elements of the processed `train_dataset` or `eval_dataset`. Will default to `DataCollatorForLanguageModeling`.

Note

`data_collator` 负责将若干条样本整理成一个Batch，并完成Padding和标签构造。

- **train_dataset** (`torch.utils.data.Dataset` | `torch.utils.data.IterableDataset` | `datasets.Dataset`, *optional*) — The dataset to use for training. If it is a `Dataset`, columns not accepted by the `model.forward()` method are automatically removed. Note that if it's a `torch.utils.data.IterableDataset` with some randomization and you are training in a distributed fashion, your iterable dataset should either use an internal attribute `generator` that is a `torch.Generator` for the randomization that must be identical on all processes (and the Trainer will manually set the seed of this `generator` at each epoch) or have a `set_epoch()` method that internally sets the seed of the RNGs used.

Note

支持的具体数据格式有：

```
json
{
  "text": "The sky is blue."
}
```

```
json
{
  "prompt": "The sky is",
  "completion": " blue."
}
```

```
json
{
  "messages": [
    {
      "role": "user",
      "content": "What color is the sky?"
    },
    {
      "role": "assistant",
      "content": "It is blue."
    }
  ]
}
```

对话格式的数据，`SFTTrainer` 会自动应用模型的Chat Template，并完成Token化和标签构造。

- `eval_dataset` (`torch.utils.data.Dataset` | `dict[str, torch.utils.data.Dataset]` | `datasets.Dataset`, *optional*) — The dataset to use for evaluation. If it is a `Dataset`, columns not accepted by the `model.forward()` method are automatically removed. If it is a dictionary, it will evaluate on each dataset prepending the dictionary key to the metric name.
- `processing_class` (`PreTrainedTokenizerBase` or `ProcessorMixin`, *optional*) — Processing class used to process the data. If provided, will be used to automatically process the inputs for the model, and it will be saved along the model to make it easier to rerun an interrupted training or reuse the fine-tuned model.
- `peft_config` (`PeftConfig`, *optional*) — PEFT configuration used to wrap the model.

Note

全参数微调时不传入 `peft_config`；LoRA和QLoRA微调时可以传入 `LoraConfig`，由 `SFTTrainer` 为基础模型注入LoRA模块。
如果传入的模型已经是可训练的 `PeftModel`，则不应再次传入该参数。

SFTConfig Parameters

`SFTConfig` 继承自 `TrainingArguments`。以下参数的类型和默认值均以 `SFTConfig` 为准。

SFT Parameters

- **max_length** (`int` or `None`, *optional*, defaults to 1024) — Maximum length of the tokenized sequence.

Note

`max_length` 用于限制单条训练序列Token化后的最大长度。超过该长度的序列会被截断；设置为 `None` 时不进行截断。

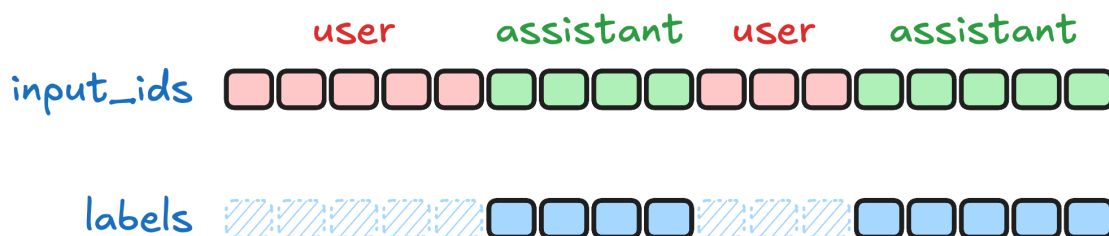
序列越长，能够保留的上下文越多，但训练所需的显存和计算量也越大。

- **assistant_only_loss** (`bool`, *optional*, defaults to `False`) — Whether to compute loss only on the assistant part of the sequence.

Note

设置为 `True` 后，只有Assistant回答对应的Token参与损失计算，System和User消息只作为输入上下文。

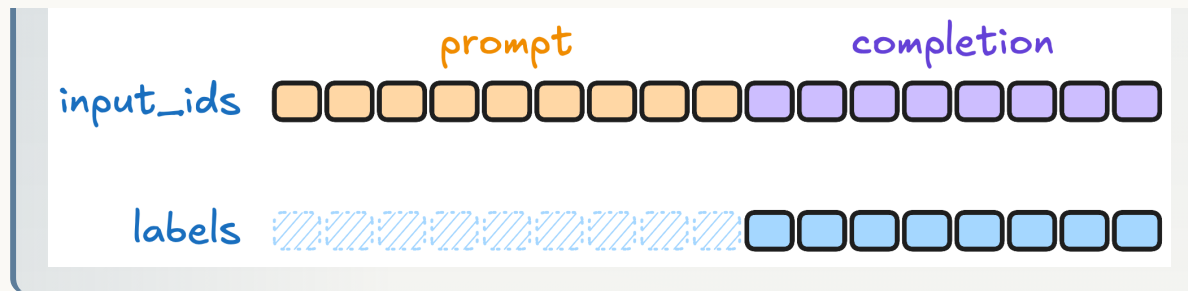
该参数仅适用于对话数据，并要求模型的Chat Template能够生成Assistant Token对应的掩码。



- **completion_only_loss** (`bool`, *optional*) — Whether to compute loss only on the completion part of the sequence. If set to `True`, loss is computed only on the completion, which is supported only for prompt-completion datasets. If `False`, loss is computed on the entire sequence. If `None` (default), the behavior depends on the dataset: loss is computed on the completion for prompt-completion datasets, and on the full sequence for language modeling datasets.

Note

设置为 `True` 后，只有completion对应的Token参与损失计算，该参数仅适用于prompt-completion数据。



Output Directory

- **output_dir** (`str` or `None`, *optional*, defaults to `None`) — The output directory where the model predictions and checkpoints will be written.

Training Duration and Batch Size

- **per_device_train_batch_size** (`int`, *optional*, defaults to 8) — The batch size *per device*. The **global batch size** is computed as: `per_device_train_batch_size * number_of_devices` in multi-GPU or distributed setups.
- **num_train_epochs** (`float`, *optional*, defaults to 3.0) — Total number of training epochs to perform (if not an integer, will perform the decimal part percents of the last epoch before stopping training).
- **max_steps** (`int`, *optional*, defaults to -1) — Overrides `num_train_epochs`. If set to a positive number, the total number of training steps to perform. For a finite dataset, training is reiterated through the dataset (if all data is exhausted) until `max_steps` is reached.

Learning Rate & Scheduler

- **learning_rate** (`float`, *optional*, defaults to 2e-5) — The initial learning rate for the optimizer. This is typically the peak learning rate when using a scheduler with warmup.
- **lr_scheduler_type** (`str` or `SchedulerType`, *optional*, defaults to `"linear"`) — The learning rate scheduler type to use. See `SchedulerType` for all possible values.

Common choices:

- `"linear"` = `get_linear_schedule_with_warmup()`
- `"cosine"` = `get_cosine_schedule_with_warmup()`
- `"constant"` = `get_constant_schedule()`
- `"constant_with_warmup"` = `get_constant_schedule_with_warmup()`
- **lr_scheduler_kwargs** (`dict` or `str`, *optional*, defaults to `None`) — The extra

arguments for the `lr_scheduler`. See the documentation of each scheduler for possible values.

- **warmup_steps** (`int` or `float`, *optional*, defaults to 0) — Number of steps for a linear warmup from 0 to `learning_rate`. Warmup helps stabilize training in the initial phase. Can be: An integer: exact number of warmup steps. A float in range [0, 1): interpreted as ratio of total training steps.

Optimizer

- **optim** (`str` or `training_args.OptimizerNames`, *optional*, defaults to `"adamw_torch_fused"`) — The optimizer to use. Common options:
 - `"adamw_torch"`: PyTorch's AdamW (recommended default)
 - `"adamw_torch_fused"`: Fused AdamW kernel
 - `"adamw_hf"`: HuggingFace's AdamW implementation
 - `"sgd"`: Stochastic Gradient Descent with momentum
 - `"adafactor"`: Memory-efficient optimizer for large models
 - `"adamw_8bit"`: 8-bit AdamW (requires bitsandbytes) See `OptimizerNames` for the complete list.

Note

`optim` 用于指定训练时使用的优化器。常用选项如下：

- **adamw_torch**：PyTorch官方实现的AdamW，是当前推荐的默认选项。在大多数场景下表现稳定，适合常规的Transformer模型微调。
- **adamw_torch_fused**：PyTorch的融合（Fused）AdamW实现，将多个逐元素操作合并为单个CUDA Kernel执行，减少Kernel启动开销，训练速度更快。功能与 `adamw_torch` 完全一致，适合对训练速度有要求且使用GPU的场景。需要PyTorch ≥ 2.0 。
- **adamw_hf**：Hugging Face自实现的AdamW，是早期版本的遗留选项。与 `adamw_torch` 行为基本一致，当前推荐优先使用 `adamw_torch`。
- **sgd**：带动量的随机梯度下降。无自适应学习率，对学习率设置较为敏感，通常需要配合精细的学习率调度。在Transformer模型训练中较少使用，更多见于CNN等视觉模型。
- **adafactor**：内存高效的自适应优化器。通过使用矩阵分解近似二阶矩，将优化器状态的显存占用从 $O(n)$ 降至 $O(\sqrt{n})$ ，适合显存受限时训练大模型。代价是收敛速度和最终精度可能略逊于AdamW。
- **adamw_8bit**：使用8-bit量化存储优化器状态的AdamW，由bitsandbytes

库提供。将一阶矩和二阶矩从FP32量化为INT8，显存占用减少约75%，同时保持与标准AdamW接近的训练效果。需要安装bitsandbytes，适合显存较为紧张的场景。

AdamW

AdamW是Adam优化器的改进版本，将权重衰减从梯度更新中解耦，使正则化更加稳定有效。

权重衰减的作用是让模型参数逐渐变小，使模型更加平滑、简化，从而减少过拟合。传统上，这一效果通过在损失函数中加入L2正则项来实现：

$$L_{\text{reg}}(w) = L(w) + \frac{\lambda}{2} \|w\|^2$$

对该损失函数对 w 求偏导可得：

$$g_t = \frac{\partial L}{\partial w} + \lambda w$$

在SGD中，参数更新为：

$$w_t = w_{t-1} - \eta \frac{\partial L}{\partial w} - \eta \lambda w_{t-1}$$

其中 $-\eta \lambda w_{t-1}$ 正是权重衰减项。因此在SGD中，L2正则化能够稳定地产生权重衰减效果。

然而在Adam中，直接加入L2正则化并不能得到正确的权重衰减效果，因为Adam会对梯度进行自适应缩放。

将 $g_t = \frac{\partial L}{\partial w} + \lambda w$ 代入Adam的更新过程：

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

经偏差校正后：

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

参数更新为：

$$w_t = w_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

由于L2项 λw 与普通梯度一同被自适应缩放，不同参数的衰减幅度也会不一致，导致正则化效果弱化甚至失效。因此，传统的“Adam + L2正则化”并不能实现真正意义上的权重衰减。

为解决这一问题，AdamW将权重衰减从梯度更新中分离出来，在参数更新阶段单独执行：

$$w_t = w_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} - \eta \lambda w_{t-1}$$

前一项是标准的Adam更新，后一项是独立的权重衰减操作，不再受梯度自适应缩放的干扰。这种方式使正则化强度稳定一致，在Transformer和大语言模型训练中取得了更好的泛化性能，成为当前主流的优化方法之一。

linear_schedule_with_warmup

常用的学习率调度策略，分为两个阶段：Warmup阶段学习率从0线性提升至设定的峰值，避免训练初期因学习率过大导致的不稳定；随后学习率从峰值线性下降至0，使模型在训练后期更平稳地收敛。这种调度方式简单可靠，在Transformer和大模型训练中十分常用。

- **weight_decay** (`float`, *optional*, defaults to 0) — Weight decay coefficient applied by the optimizer (not the loss function). Adds L2 regularization to prevent overfitting by penalizing large weights. Automatically excluded from bias and LayerNorm parameters. Typical values: 0.01 (standard), 0.1 (stronger regularization), 0.0 (no regularization).
- **adam_beta1** (`float`, *optional*, defaults to 0.9) — The exponential decay rate for the first moment estimates (momentum) in Adam-based optimizers. Controls how much history of gradients to retain.
- **adam_beta2** (`float`, *optional*, defaults to 0.999) — The exponential decay rate for the second moment estimates (variance) in Adam-based optimizers. Controls adaptive learning rate scaling.
- **adam_epsilon** (`float`, *optional*, defaults to 1e-8) — Epsilon value for numerical stability in Adam-based optimizers. Prevents division by zero in the denominator of the update rule.

Regularization & Training Stability

- **gradient_accumulation_steps** (`int`, *optional*, defaults to 1) — Number of update steps to accumulate gradients before performing a backward/update pass. Simulates larger batch sizes without additional memory. Effective batch size = `per_device_train_batch_size × num_devices × gradient_accumulation_steps`.

 Note

`gradient_accumulation_steps` 用于在显存不足以支撑大Batch Size时，通过多步累积梯度来模拟更大的有效Batch Size。设置为 k 时，Trainer会连续执行 k 次前向与反向传播，将梯度累加后再执行一次参数更新，等效Batch Size为：

$$\text{effective_batch_size} = \text{per_device_train_batch_size} \times \text{num_devices} \times k$$

- **max_grad_norm** (`float`, *optional*, defaults to 1.0) — Maximum gradient norm for gradient clipping. Applied after backward pass, before optimizer step. Prevents gradient explosion by scaling down gradients when their global norm exceeds this threshold. Set to 0 to disable clipping. Typical values: 1.0 (standard), 0.5 (more conservative), 5.0 (less aggressive).

 Note

`max_grad_norm` 用于梯度裁剪，在每次参数更新前将所有参数梯度的全局L2范数限制在该阈值以内。其计算方式为：

$$\|g\|_2 = \sqrt{\sum_i g_i^2}$$

当 $\|g\|_2 > \text{max_grad_norm}$ 时，对所有梯度按比例缩放：

$$g_i \leftarrow g_i \cdot \frac{\text{max_grad_norm}}{\|g\|_2}$$

梯度爆炸是深度网络训练中的常见问题，尤其在序列较长或网络较深时，反向传播过程中梯度可能被连续放大，导致参数更新幅度过大，训练不稳定甚至发散。梯度裁剪通过限制更新幅度来稳定训练过程，同时保持梯度的方向不变。默认值1.0适用于大多数场景，设为0则禁用裁剪。

Mixed Precision Training

 Important

混合精度训练（Mixed Precision Training）是一种在深度学习中常用的优化技术，其核心思想是在训练过程中同时使用FP16（半精度浮点数）和FP32（单精度浮点数），从而提升训练速度并减少显存占用。

传统训练通常全部采用FP32进行计算。FP32数值稳定性较好，但显存占用较高、计算速度相对较慢。而现代GPU（如NVIDIA Tensor Core）对FP16/BF16运算进行了硬件加

速，因此将部分计算切换为低精度后，可以显著提高训练效率。

不过，直接使用FP16训练可能会出现数值稳定性问题。例如在反向传播过程中，某些梯度可能非常小，而FP16能表示的数值范围有限，过小的梯度会发生下溢（underflow），最终变成0，从而影响模型训练。

下面是PyTorch中自动混合精度训练（AMP）的示例：

```
python

# 导入自动混合精度工具
from torch.cuda.amp import autocast

# 导入梯度缩放器
from torch.cuda.amp import GradScaler

# 创建梯度缩放器
scaler = GradScaler()

# 开始训练循环
for input, target in dataloader:

    # 清空上一轮梯度
    optimizer.zero_grad()

    # 开启自动混合精度环境
    with autocast():

        # 前向传播
        output = model(input)

        # 计算损失
        loss = criterion(output, target)

    # 放大loss后执行反向传播
    scaler.scale(loss).backward()

    # 更新模型参数
    scaler.step(optimizer)

    # 更新缩放因子
    scaler.update()
```

混合精度训练的优点主要包括：

- 提高训练速度；

- 减少GPU显存占用；
- 支持更大的Batch Size；
- 在大多数任务中能够保持与FP32接近的训练效果。

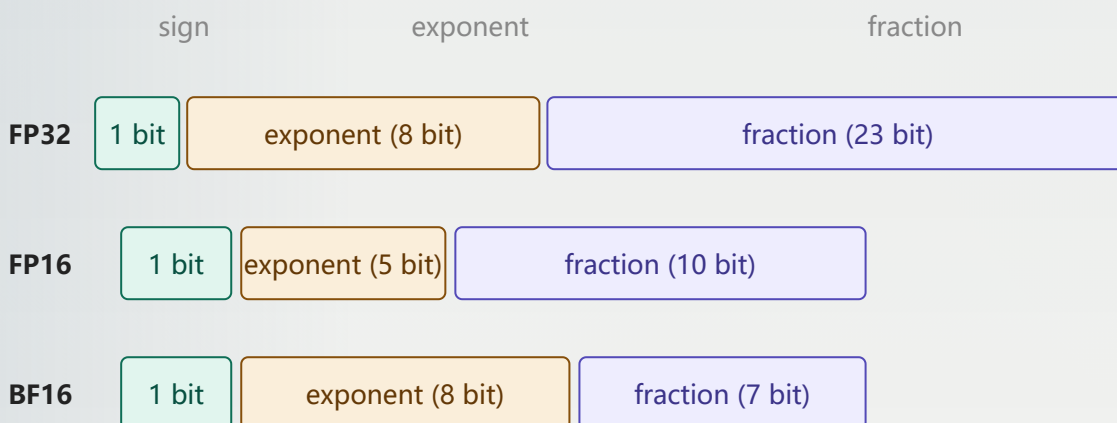
目前，混合精度训练已经广泛应用于Transformer、大语言模型、扩散模型等深度学习任务中。

- **bf16** (`bool` or `None`, *optional*, defaults to `None`) — Enable bfloat16 (BF16) mixed precision training Generally preferred over FP16 due to better numerical stability and no loss scaling required.
- **fp16** (`bool`, *optional*, defaults to `False`) — Enable float16 (FP16) mixed precision training. Consider using BF16 instead if your hardware supports it.

Note

浮点数由三部分组成：符号位（**sign**）、指数位（**exponent**）和尾数位（**fraction**）。指数位决定数值的动态范围，尾数位决定精度。

如下图所示，FP32使用8位指数和23位尾数；FP16将指数压缩至5位、尾数保留10位，动态范围变小，容易出现溢出，训练时需要配合Loss Scaling；BF16保留了与FP32相同的8位指数，仅将尾数压缩至7位，动态范围与FP32一致，数值更稳定，但精度略低于FP16。



`bf16` 和 `fp16` 均用于开启混合精度训练，二者不可同时设置为 `True`。

- **fp16**：使用FP16进行混合精度训练。由于FP16动态范围较小，训练过程中容易出现梯度下溢或数值溢出，PyTorch的自动混合精度（AMP）会自动启用Loss Scaling来稳定训练。
- **bf16**：使用BF16进行混合精度训练。BF16与FP32拥有相同的动态范围，数值更稳定，因此无需Loss Scaling。在支持BF16的硬件上优先推荐使用 `bf16`。

在未启用FP16时，`SFTConfig` 会优先使用BF16，但最终仍取决于训练设备是否支持。

- `bf16_full_eval` (`bool`, *optional*, defaults to `False`) — Use full BF16 precision for evaluation (not just mixed precision). Faster and saves memory but may affect metric values slightly. Only applies during evaluation.
- `fp16_full_eval` (`bool`, *optional*, defaults to `False`) — Use full FP16 precision for evaluation (not just mixed precision). Faster and saves memory but may affect metric values slightly. Only applies during evaluation.

Note

`bf16_full_eval` 和 `fp16_full_eval` 分别控制评估阶段是否使用全量BF16或FP16精度，而不是混合精度。开启后评估速度更快、显存占用更低，但评估指标可能与FP32结果存在细微差异。这两个参数仅影响评估过程。

Gradient Checkpointing

- `gradient_checkpointing` (`bool`, *optional*, defaults to `True`) — Enable gradient checkpointing to trade compute for memory. Reduces memory usage by clearing activations during forward pass and recomputing them during backward pass. Enables training larger models or batch sizes at the cost of ~20% slower training.

Note

Gradient Checkpointing用于在显存不足时以计算换显存。

标准训练中，前向传播会保存所有层的激活值，供反向传播计算梯度时使用，显存占用随网络深度线性增长。开启梯度检查点后，前向传播不再保存所有层的激活值，反向传播需要某层的激活值时，会从最近的检查点重新执行一次前向计算来还原它。这样显存占用大幅降低，代价是额外计算开销。

标准训练

Layer 1

保存

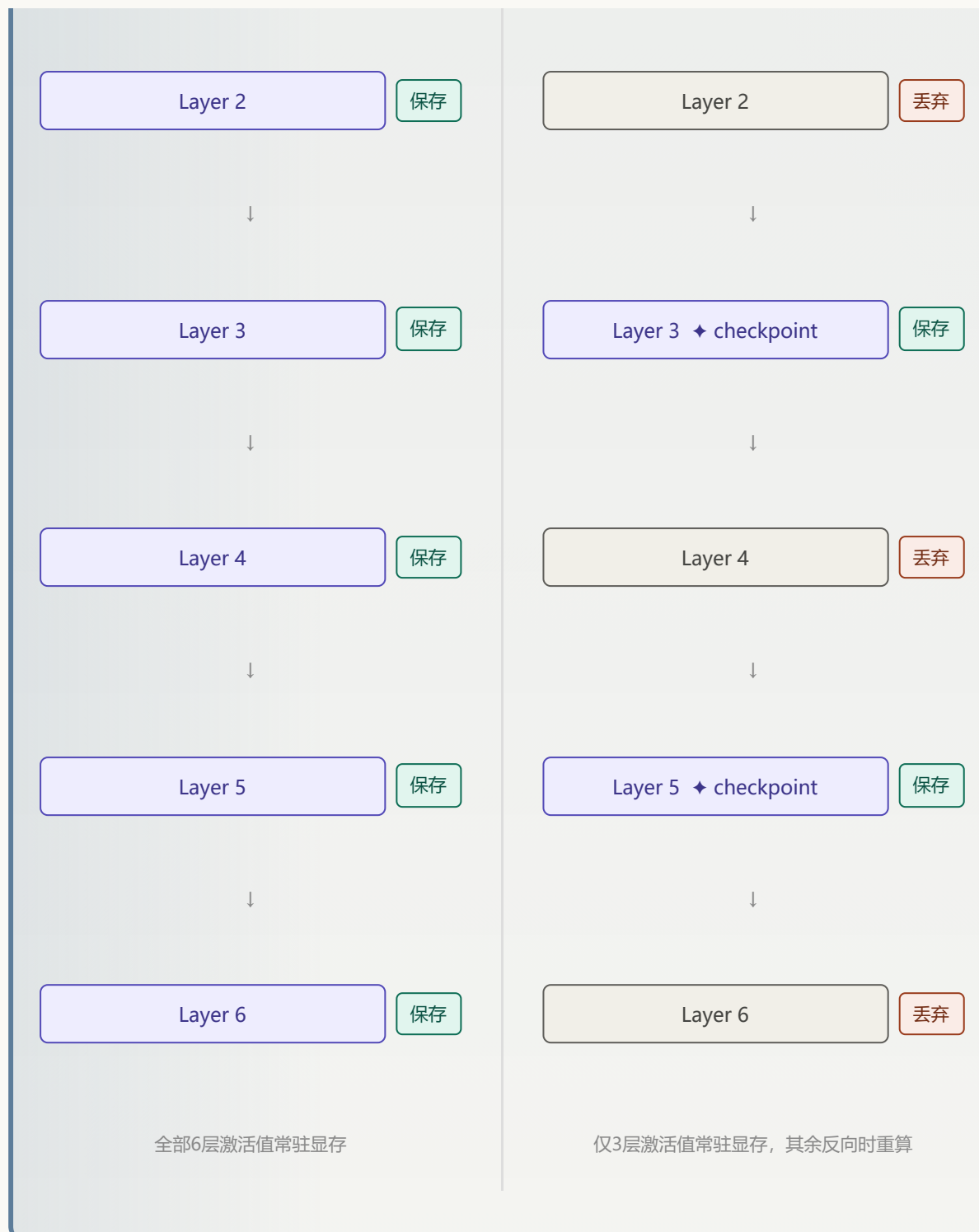


梯度检查点

Layer 1 → checkpoint

保存





Logging & Monitoring Training

- **report_to** (`str` or `list[str]`, *optional*, defaults to `"none"`) — The list of integrations to report the results and logs to. Supported platforms are `"azure_ml"`, `"clearml"`, `"codecarbon"`, `"comet_ml"`, `"dagshub"`, `"dvclive"`, `"flyte"`, `"mlflow"`, `"swanlab"`, `"tensorboard"`, `"trackio"` and `"wandb"`. Use `"all"` to report to all integrations installed, `"none"` for no integrations.
- **logging_strategy** (`str` or `IntervalStrategy`, *optional*, defaults to `"steps"`) — The

logging strategy to adopt during training. Possible values are:

- `"no"`: No logging is done during training.
 - `"epoch"`: Logging is done at the end of each epoch.
 - `"steps"`: Logging is done every `logging_steps`.
- `logging_steps` (`int` or `float`, *optional*, defaults to 10) — Number of update steps between two logs if `logging_strategy="steps"`. Should be an integer or a float in range `[0,1)`. If smaller than 1, will be interpreted as ratio of total training steps.

Evaluation

- `eval_strategy` (`str` or `IntervalStrategy`, *optional*, defaults to `"no"`) — When to run evaluation. Options:
 - `"no"`: No evaluation during training
 - `"steps"`: Evaluate every `eval_steps`
 - `"epoch"`: Evaluate at the end of each epoch
- `eval_steps` (`int` or `float`, *optional*) — Number of update steps between two evaluations if `eval_strategy="steps"`. Will default to the same value as `logging_steps` if not set. Should be an integer or a float in range `[0,1)`. If smaller than 1, will be interpreted as ratio of total training steps.
- `per_device_eval_batch_size` (`int`, *optional*, defaults to 8) — The batch size per device accelerator core/CPU for evaluation.

Checkpointing & Saving

- `save_only_model` (`bool`, *optional*, defaults to `False`) — Save only model weights, not optimizer/scheduler/RNG state. Significantly reduces checkpoint size but prevents resuming training from the checkpoint. Use when you only need the trained model for inference, not continued training. You can only load the model using `from_pretrained` with this option set to `True`.
- `save_strategy` (`str` or `SaveStrategy`, *optional*, defaults to `"steps"`) — The checkpoint save strategy to adopt during training. Possible values are:
 - `"no"`: No save is done during training.
 - `"epoch"`: Save is done at the end of each epoch.
 - `"steps"`: Save is done every `save_steps`.
 - `"best"`: Save is done whenever a new `best_metric` is achieved.
- `save_steps` (`int` or `float`, *optional*, defaults to 500) — Number of updates steps before two checkpoint saves if `save_strategy="steps"`. Should be an integer or a

float in range `[0,1)`. If smaller than 1, will be interpreted as ratio of total training steps.

- **save_total_limit** (`int`, *optional*) — Maximum number of checkpoints to keep. Deletes older checkpoints in `output_dir`. When `load_best_model_at_end=True`, the best checkpoint is always retained plus the most recent ones. For example, `save_total_limit=5` keeps the 4 most recent plus the best.

Best Model Tracking

- **load_best_model_at_end** (`bool`, *optional*, defaults to `False`) — Load the best checkpoint at the end of training. Requires `eval_strategy` to be set. When enabled, the best checkpoint is always saved (see `save_total_limit`). When `True`, `save_strategy` must match `eval_strategy` (unless `save_strategy` is `"best"`), and if using `"steps"`, `save_steps` must be a multiple of `eval_steps`.
- **metric_for_best_model** (`str`, *optional*) — Metric to use for comparing models when `load_best_model_at_end=True`. Must be a metric name returned by evaluation, with or without the `"eval_"` prefix. Defaults to `"loss"`. If you set this, `greater_is_better` will default to `True` unless the name ends with `"loss"`. Examples: `"accuracy"`, `"f1"`, `"eval_bleu"`.
- **greater_is_better** (`bool`, *optional*) — Whether higher metric values are better. Defaults based on `metric_for_best_model`: `True` if the metric name doesn't end in `"loss"`, `False` otherwise.